



Operation Reshearch

Projet : Flots maximum, coupes minimales et flots de coût minimum

Étudiant : Jules Stevenson

Email : stevenson.jules@etu.univ-cotedazur.fr

Encadrant : Jean-Charles Régim

Formation : M1 Informatique DS4H

Année : 2025–2026

May 9, 2026

Contents

1	Présentation	1
2	Structure de données du graphe résiduel	1
2.1	Rappel : qu'est-ce que le graphe résiduel ?	1
2.2	Choix d'implémentation	1
2.2.1	Représentation par paires d'arcs liés (Edge + reverse)	1
2.2.2	Liste d'adjacence sur le résiduel	2
2.3	Invariants maintenus	2
2.4	Reset du flot	3
3	Algorithme de Ford-Fulkerson variante Edmonds-Karp	3
3.1	Idée	3
3.2	Pourquoi BFS (Edmonds-Karp) plutôt que DFS (Ford-Fulkerson original) ?	3
3.3	Coupe minimale	3
4	Détection de cycles de coût négatif	3
4.1	Pourquoi on en a besoin	3
4.2	Comment	4
5	Min-cost flow par chemins augmentants	4
5.1	Schéma général (Successive Shortest Paths)	4
5.2	Variante 2.1 chemin de coût minimum par Bellman-Ford	4
5.3	Variante 2.2 chemin de coût minimum par Dijkstra avec potentiels	5
5.3.1	Pourquoi cette variante ?	5
5.3.2	La transformation	5
5.3.3	Initialisation des potentiels	5
5.3.4	Mise à jour des potentiels après une itération	5
5.4	Vérification des deux variantes	6
6	Tests effectués et résultats	6
6.1	Test 1 Max flow simple (DearStudents.docx, exemple 1)	6
6.2	Test 2 Min-cost flow (DearStudents.docx, exemple 3)	6
6.3	Test 3 Assignment 10 personnes / 8 tâches	6
6.4	Test 4 Format prof (graph_data.txt)	6
6.5	Test 5 Détection de cycle négatif	7
7	Complexités	7
8	Conclusion	7

1 Présentation

L'objectif est d'implémenter, **sans aucune librairie de flot** (NetworkX, JGraphT, etc.), trois familles d'algorithmes sur les graphes orientés capacité-coût :

1. Le calcul du **flot maximum** par Ford-Fulkerson (variante Edmonds-Karp), avec extraction de la **coupe minimale**.
2. Le calcul du **flot de coût minimum** par chemins augmentants successifs (Successive Shortest Paths, SSP) en deux variantes :
 - chemin de coût minimum trouvé par Bellman-Ford ;
 - chemin de coût minimum trouvé par Dijkstra avec **potentiels** (renormalisation à la Johnson).
3. La **détection de cycles de coût négatif** dans le graphe résiduel, utilisée comme garde-fou : on prouve et on vérifie que l'algorithme SSP n'en crée jamais.

Tout le code est en Java 21. La conception donne la priorité à la lisibilité du code (POO, pas d'astuce d'indexation) tout en gardant des complexités optimales.

2 Structure de données du graphe résiduel

C'est le cœur du projet. Une fois cette structure correcte, tous les algorithmes s'écrivent quasi mécaniquement.

2.1 Rappel : qu'est-ce que le graphe résiduel ?

Étant donné un graphe orienté capacité-coût $G = (V, E)$ avec, pour chaque arc $(u, v) \in E$, une capacité $c(u, v) \geq 0$ et un coût unitaire $k(u, v)$, et étant donné un flot f sur G , le *graphe résiduel* G_f est défini ainsi :

- pour chaque arc (u, v) de G avec $f(u, v) < c(u, v)$, on met dans G_f un arc **forward** (u, v) de capacité résiduelle $c(u, v) - f(u, v)$ et de coût $+k(u, v)$; « on peut encore envoyer du flot dans le sens d'origine » ;
- pour chaque arc (u, v) de G avec $f(u, v) > 0$, on met dans G_f un arc **backward** (v, u) de capacité résiduelle $f(u, v)$ et de coût $-k(u, v)$; « on peut défaire du flot déjà envoyé ».

Le coût $-k$ de l'arc inverse est imposé : si l'on a payé k pour envoyer une unité de u à v , l'annuler doit « rendre » k au coût total. Sans ce signe, le coût total du flot ne serait pas conservé sur les modifications symétriques.

2.2 Choix d'implémentation

2.2.1 Représentation par paires d'arcs liés (Edge + reverse)

J'ai choisi la représentation classique où **chaque arc original est stocké comme deux objets Edge** :

- un Edge *forward* de u à v , avec `capacity = c`, `cost = +k`, `initialCapacity = c` ;
- un Edge *backward* de v à u , avec `capacity = 0`, `cost = -k`, `initialCapacity = 0`.

Les deux objets se référencent mutuellement par un champ Edge `reverse`. Cela permet, à partir d'un arc, de retrouver son symétrique en **temps constant**, opération critique parce que toute augmentation de flot doit mettre à jour les deux capacités résiduelles simultanément.

```

1 public class Edge {
2     public final int from, to;
3     public final long cost;
4     public final long initialCapacity;
5     public long capacity;           // capacité résiduelle ACTUELLE
6     public Edge reverse;           // arc inverse, liée à la création
7 }

```

Alternative considérée et rejetée : la représentation « en tableau plat » utilisée typiquement en C/C++ (les arcs sont ajoutés deux par deux dans un tableau, et l'inverse de l'arc d'index i est l'arc d'index $i \oplus 1$). Plus compacte, mais peu lisible et peu Java-idiomatique. La référence directe `reverse` est strictement équivalente en complexité, et bien plus claire à expliquer.

2.2.2 Liste d'adjacence sur le résiduel

Le graphe est représenté par `List<List<Edge>> adj`, où `adj.get(u)` contient *tous* les arcs résiduels sortant de u ; à la fois les forwards d'arcs originaux partant de u , et les backwards d'arcs originaux arrivant à u . C'est volontaire : les algorithmes (BFS dans Ford-Fulkerson, relaxation dans Bellman-Ford, expansion dans Dijkstra) ne distinguent jamais forward et backward ; ils ne demandent qu'une chose : « depuis u , où puis-je aller, avec quelle capacité résiduelle et à quel coût ? »

Concrètement, `Graph.addEdge(u, v, c, k)` :

1. Crée le forward et le backward.
2. Les lie via `forward.reverse = backward`; `backward.reverse = forward`.
3. Insère le forward dans `adj.get(u)` et le backward dans `adj.get(v)`.

Du point de vue des algorithmes, le graphe résiduel *est* ce graphe, à un filtrage près sur `capacity > 0`.

2.3 Invariants maintenus

Les propriétés suivantes sont maintenues à tout instant après initialisation :

1. **Conservation locale par paire d'arcs :**
`forward.capacity + backward.capacity = forward.initialCapacity` (constant).
2. **Flot actuel** sur l'arc original = `backward.capacity = forward.reverse.capacity`. Cela donne la méthode `flow()` qui ne fait que renvoyer `reverse.capacity`.
3. **Symétrie des coûts :** `backward.cost = -forward.cost`.
4. **isForward()** distingue les arcs « réels » (ceux qui apparaissaient dans le graphe d'origine) des arcs introduits artificiellement par le résiduel : `initialCapacity > 0`.
5. **Augmentation atomique :** la méthode `pushFlow(amount)` garantit que `capacity` du forward et `capacity` du backward bougent toujours de manière cohérente.

```

1 public void pushFlow(long amount) {
2     this.capacity -= amount;
3     this.reverse.capacity += amount;
4 }

```

Ces cinq invariants suffisent à expliquer l'intégralité du comportement de la structure pendant toute la durée d'exécution des algorithmes ci-dessous.

2.4 Reset du flot

Pour pouvoir relancer un autre algorithme sur le même graphe sans devoir reconstruire la structure, `Graph.resetFlow()` repositionne, pour chaque arc original : `forward.capacity = initialCapacity` et `backward.capacity = 0`. Cela évite des tests biaisés par l'état laissé par l'algorithme précédent.

3 Algorithme de Ford-Fulkerson variante Edmonds-Karp

3.1 Idée

Tant qu'il existe un chemin de s à t dans le résiduel (un *chemin augmentant*), on en trouve un, on calcule le minimum des capacités résiduelles le long du chemin (le *bottleneck*), on pousse cette quantité de flot, et on recommence. À la fin, le flot est maximum.

3.2 Pourquoi BFS (Edmonds-Karp) plutôt que DFS (Ford-Fulkerson original) ?

Avec DFS, on peut tomber sur des chemins très longs et faire un nombre d'itérations qui dépend des *valeurs* des capacités (cf. l'exemple-jouet à 4 nœuds avec capacité 1000 où DFS fait 2000 itérations). Avec BFS, on prend toujours le chemin avec **le moins d'arcs**, ce qui borne le nombre d'augmentations à $O(V \cdot E)$. Le coût d'un BFS étant $O(E)$, la complexité totale est $O(V \cdot E^2)$. C'est strictement meilleur asymptotiquement, et l'implémentation est tout aussi simple.

3.3 Coupe minimale

À la fin de l'algorithme, on calcule l'ensemble S des sommets atteignables depuis s dans le résiduel par un BFS/DFS. Par le **théorème max-flow min-cut**, la coupe $(S, V \setminus S)$ est minimale et sa capacité est exactement la valeur du flot maximum. Les arcs qui constituent la coupe sont les arcs originaux (u, v) avec $u \in S$ et $v \notin S$; chacun d'eux est nécessairement saturé par le flot final.

4 Détection de cycles de coût négatif

4.1 Pourquoi on en a besoin

L'énoncé exige : « *ne jamais avoir de cycles négatifs* ». L'argument est le suivant : si à un moment le résiduel contient un cycle de coût négatif, alors le flot courant **n'est pas optimal** ; on pourrait envoyer du flot le long de ce cycle pour diminuer le coût total sans changer la valeur du flot (c'est l'algorithme de Klein de *cycle-canceling*).

Le théorème d'optimalité de Klein dit : *un flot est de coût minimum si et seulement si son résiduel ne contient aucun cycle de coût négatif*. L'algorithme SSP exploite cette propriété en maintenant l'invariant à chaque itération.

Pour le projet, on implémente :

- la détection elle-même comme algorithme autonome (`BellmanFord.findNegativeCycle`) ;
- une vérification systématique après chaque augmentation dans `MinCostFlow` (paramètre `checkInvariant = true` par défaut), qui lève une `AssertionError` si jamais un cycle apparaît, ce qui n'arrive jamais sur les exemples du projet, prouvant empiriquement la correction de SSP.

4.2 Comment

L'algorithme est une variante de Bellman-Ford. La subtilité : on veut détecter un cycle de coût négatif *n'importe où* dans le graphe (pas seulement accessible depuis un sommet particulier). On utilise pour cela l'astuce suivante :

On initialise $\text{dist}[v] = 0$ pour *tout* v , au lieu de $\text{dist}[s] = 0$ et $\text{dist}[\text{autres}] = +\infty$.

C'est équivalent à ajouter une « super-source » virtuelle reliée à chaque sommet par un arc de coût 0, donc tout cycle négatif est désormais accessible depuis cette super-source. On applique alors le critère standard : on relaxe tous les arcs V fois ; si à la V -ième passe une distance change encore, c'est qu'il existe un cycle négatif accessible, et donc, par construction, un cycle négatif tout court.

Pour **récupérer** le cycle (et pas seulement détecter son existence), on retient le dernier sommet **last** dont la distance a été relaxée à la V -ième passe ; on remonte **parent[last]** pendant V étapes (ce qui garantit d'entrer dans le cycle) ; puis on remonte les parents jusqu'à revenir au point d'entrée. Cela donne la liste des arcs du cycle, qu'on peut afficher pour vérification.

Complexité : $O(V \cdot E)$.

5 Min-cost flow par chemins augmentants

5.1 Schéma général (Successive Shortest Paths)

```
flot <- 0 ; cout <- 0
tant qu'il existe un chemin de s à t dans le résiduel :
  P <- chemin de COUT MINIMUM de s à t dans le résiduel
  b <- min des capacités résiduelles le long de P
  pousser b unités de flot le long de P
  flot <- flot + b
  cout <- cout + b * (cout de P)
```

À chaque itération, on pousse le flot sur le chemin le moins cher. Le théorème central est :

Si à chaque étape P est un chemin de coût minimum de s à t dans le résiduel, alors le résiduel obtenu après augmentation ne contient aucun cycle de coût négatif.

C'est exactement l'invariant qu'on souhaite maintenir. Initialement, le flot nul a un résiduel = graphe d'origine, qui est sans cycle négatif (les coûts sont supposés ≥ 0 dans le projet) ; après augmentation par un chemin de coût minimum, l'invariant est préservé. Par récurrence, à toutes les itérations, l'invariant tient, ce que la vérification systématique confirme.

5.2 Variante 2.1 chemin de coût minimum par Bellman-Ford

C'est la version directe : à chaque itération, on lance Bellman-Ford depuis s sur le résiduel, qui renvoie les distances minimales $\text{dist}[v]$ (et le tableau **parentEdge** pour reconstruire le chemin). Bellman-Ford est nécessaire (et pas Dijkstra) parce que le résiduel contient des arcs **de coût négatif** (les arcs inverses).

Complexité par itération : $O(V \cdot E)$. Nombre d'itérations : borné par $O(V \cdot E)$ ou par la valeur du flot dans le pire cas.

5.3 Variante 2.2 chemin de coût minimum par Dijkstra avec potentiels

5.3.1 Pourquoi cette variante ?

Bellman-Ford coûte $O(V \cdot E)$ par itération. Dijkstra coûte $O((V + E) \log V)$ avec une file de priorité, ce qui est asymptotiquement bien meilleur sur les graphes peu denses. Mais Dijkstra est faux en présence d'arcs de coût négatif. L'astuce de Johnson permet de **renormaliser** les coûts pour qu'ils soient tous positifs, sans changer les plus courts chemins.

5.3.2 La transformation

On maintient un vecteur $\varphi : V \rightarrow \mathbb{R}$ de **potentiels**. À partir d'un arc (u, v) de coût $c(u, v)$, on définit le **coût réduit** :

$$c'(u, v) = c(u, v) + \varphi(u) - \varphi(v)$$

Propriété 1 (somme télescopique). Pour tout chemin $P : s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = t$,

$$\sum_i c'(v_i, v_{i+1}) = \sum_i [c(v_i, v_{i+1}) + \varphi(v_i) - \varphi(v_{i+1})] = \left(\sum_i c(v_i, v_{i+1}) \right) + \varphi(s) - \varphi(t).$$

Le terme $\varphi(s) - \varphi(t)$ est constant pour tous les chemins de s à t . Donc **minimiser le coût réduit revient à minimiser le coût original** : Dijkstra sur les coûts réduits trouve bien le chemin de coût minimum réel.

Propriété 2 (positivité, si on choisit bien φ). Si $\varphi(v)$ est la distance minimale de s à v dans le graphe initial, alors par l'inégalité triangulaire des plus courts chemins, pour tout arc (u, v) on a $\varphi(v) \leq \varphi(u) + c(u, v)$, donc $c'(u, v) \geq 0$. Dijkstra est applicable.

5.3.3 Initialisation des potentiels

Dans l'implémentation, j'initialise φ par **un seul Bellman-Ford** lancé depuis s au début de l'algorithme (cela gère le cas général, y compris celui où le graphe original aurait des coûts négatifs). Si on suppose que les coûts originaux sont tous ≥ 0 , comme c'est le cas dans tous les exemples du projet, on pourrait aussi initialiser $\varphi \equiv 0$, qui satisfait directement la propriété 2.

5.3.4 Mise à jour des potentiels après une itération

Après l'augmentation, des nouveaux arcs résiduels apparaissent (notamment les backward correspondant à l'augmentation), avec des coûts qui pourraient invalider la positivité. La règle de mise à jour est :

Après un Dijkstra qui a calculé $\text{dist}[v]$ (en coûts réduits) depuis s , on pose $\varphi(v) \leftarrow \varphi(v) + \text{dist}[v]$.

Démonstration que les coûts réduits restent ≥ 0 sur tous les arcs résiduels (y compris les nouveaux backward) : sur un arc (u, v) du chemin emprunté, par optimalité de Dijkstra on a $\text{dist}[v] = \text{dist}[u] + c'_{\text{old}}(u, v)$, donc $\text{dist}[v] - \text{dist}[u] = c'_{\text{old}}(u, v)$, ce qui implique que le nouveau coût réduit

$$c'_{\text{new}}(u, v) = c(u, v) + \varphi_{\text{new}}(u) - \varphi_{\text{new}}(v) = c'_{\text{old}}(u, v) - (\text{dist}[v] - \text{dist}[u]) = 0.$$

Sur l'arc inverse (v, u) , qui devient résiduel après l'augmentation; le coût réduit est aussi 0. Sur les arcs hors chemin, on a $\text{dist}[v] \leq \text{dist}[u] + c'_{\text{old}}(u, v)$, donc le nouveau coût reste ≥ 0 . Q.E.D.

L'implémentation contient un `IllegalStateException` si jamais un coût réduit négatif était observé pendant Dijkstra : c'est un sanity check qui confirme que les potentiels sont bien maintenus.

Complexité par itération : $O((V + E) \log V)$.

5.4 Vérification des deux variantes

Sur tous les exemples testés, les deux variantes renvoient **strictement le même flot et le même coût** (à la valeur du flot près, au coût près, et arc-par-arc dans la majorité des cas, modulo des optima alternatifs lorsqu'il en existe).

6 Tests effectués et résultats

6.1 Test 1 Max flow simple (DearStudents.docx, exemple 1)

5 sommets, 9 arcs ; source = 0, puits = 4 ; capacités allant de 5 à 40.

- **Flot maximum trouvé : 60.**
- **Coupe minimale :** $S = \{0\}$, c.-à-d. les arcs sortant directement de la source : $(0, 1)$ capa 20 + $(0, 2)$ capa 30 + $(0, 3)$ capa 10 = 60. Cohérent avec max-flow min-cut.

6.2 Test 2 Min-cost flow (DearStudents.docx, exemple 3)

7 sommets (les 5 originaux + s ajouté + t ajouté). On ajoute $s \rightarrow 0$ capa 20 coût 0, $3 \rightarrow t$ capa 5 coût 0, $4 \rightarrow t$ capa 15 coût 0.

- Min-cost MAX-flow de s à t : **flot** = 20, **coût** = 150, en 5 itérations, identique pour Bellman-Ford et Dijkstra+potentiels.
- Variante de 0 à 4 sur le graphe d'origine : **flot** = 23, **coût** = 187, en 4 itérations.

6.3 Test 3 Assignment 10 personnes / 8 tâches

- Cas sans capacité de tâche : chaque personne est assignée à la tâche la moins chère pour elle (aucune contrainte d'unicité côté tâche). Coût total optimal : **370**.
- Cas avec capacité de tâche = 2 : chaque tâche peut être prise au plus deux fois. Coût total optimal : **404**.
- Cas « chaque tâche au moins une fois » : non implémenté tel quel, il nécessite une transformation du problème pour gérer des **bornes inférieures** sur les arcs tâche $\rightarrow t$. Approche classique (Ahuja, Magnanti & Orlin) : pour chaque arc avec borne inf ℓ , on envoie d'abord ℓ unités forcées en amenant le flot au sommet d'amont par une super-source dédiée et au sommet d'aval par un super-puits, puis on lance min-cost max-flow sur le résidu. Hors scope strict du sujet.

6.4 Test 4 Format prof (graph_data.txt)

6 sommets, 10 arcs, source = 5, puits = 4.

- **Max flow : 15** ; min cut = $(0 \rightarrow 2) + (1 \rightarrow 3)$, capacité $5 + 10 = 15$.
- **Min-cost max-flow : 149** (4 itérations), identique pour les deux variantes.
- **Vérification d'invariant** : aucun cycle négatif dans le résiduel à la fin (et la vérification activée à chaque itération de SSP n'a jamais détecté de violation).

6.5 Test 5 Détection de cycle négatif

- Sur un graphe-jouet $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ avec coûts tous à -1 : le cycle est correctement détecté, et son coût (-3) est correctement reconstitué.
- Sur le même graphe avec coûts $+1$: aucun cycle détecté (correct).

7 Complexités

Algorithme	Complexité (par exécution complète)
Ford-Fulkerson (Edmonds-Karp)	$O(V \cdot E^2)$
Détection de cycle négatif	$O(V \cdot E)$
SSP via Bellman-Ford	$O(F \cdot V \cdot E)$ ou $O(V \cdot E^2)$, selon ce qui borne le nombre d'itérations
SSP via Dijkstra + potentiels	$O(F \cdot (V + E) \log V)$, avec un Bellman-Ford initial en $O(V \cdot E)$

Où F désigne la valeur du flot envoyé. Les complexités peuvent être affinées en bornant F par la coupe minimale ou par la somme des capacités sortant de la source.

8 Conclusion

Le projet illustre l'unité algorithmique des problèmes de flot : une fois posée correctement la structure du graphe résiduel, Ford-Fulkerson, Klein, et SSP (en deux variantes) s'expriment comme de simples variations sur la même boucle « *trouver un chemin / un cycle, augmenter, recommencer* ». Le travail a confirmé empiriquement le théorème de Klein (jamais de cycle négatif observé après une augmentation correcte) et la cohérence des deux variantes de SSP. Le code total fait environ 700 lignes Java commentées, sans dépendance externe.